

Monitoreo automatizado de la salud del sistema con cron

Práctica: **Crontab**

Materia: Sistemas Operativos

Alumno: Héctor _____

Docente: _____

Fecha: _____

1. Introducción

Al usar Ubuntu Desktop para estudiar o programar es común mantener abiertos al mismo tiempo el navegador con muchas pestañas, reproductores de música, editores de código y varias terminales. Cada uno de estos programas crea uno o más procesos que reservan memoria y, en algunos casos, escriben cachés y archivos temporales en disco. Sin una herramienta de observación, el usuario solo percibe el problema cuando el sistema se vuelve lento o aparece el aviso de disco lleno.

En esta práctica se construyó un script en Bash que registra el estado de la RAM, la swap, el disco de la carpeta personal, la carga del procesador y los procesos que más memoria consumen. Se programó con `cron` para que se ejecute en segundo plano cada 2 minutos y guarde un historial en `~/salud_sistema/salud_sistema.log`.

2. Objetivos

- Comprender el funcionamiento de `cron` como demonio planificador de tareas del sistema operativo.
- Automatizar la recolección de métricas del sistema mediante un script Bash comentado y robusto.
- Observar y justificar teóricamente cómo el sistema operativo administra memoria, almacenamiento y CPU cuando hay múltiples aplicaciones de escritorio abiertas.

3. Marco teórico

3.1 Demonios y `cron`

Un demonio es un proceso que se ejecuta en segundo plano, sin terminal asociada, y ofrece un servicio al sistema. `cron` es iniciado por `systemd` al arrancar. Cada minuto despierta, lee las tablas de tareas (*crontabs*) de los usuarios y del sistema, y crea (`fork + exec`) un proceso por cada tarea cuya expresión coincida con la hora actual. Terminada la tarea, el proceso hijo finaliza y libera sus recursos.

La expresión usada, `* / 2 * * * *`, tiene cinco campos: minuto, hora, día del mes, mes y día de la semana. `* / 2` en el campo de minutos significa “cada minuto divisible entre 2”.

3.2 Gestión de memoria en Linux

Linux usa memoria virtual: cada proceso cree tener su propio espacio de direcciones y el kernel traduce esas direcciones a marcos físicos de RAM. La RAM que no usan las aplicaciones no se deja ociosa; se ocupa como *page cache* (caché de disco) y se libera cuando un proceso la necesita. Por eso `free` distingue entre *used* y *available*. Esta última estima cuánta memoria pueden obtener las aplicaciones sin recurrir a swap, y es la que se usa en el script para calcular el porcentaje real de uso.

Cuando la memoria disponible se agota, el kernel mueve páginas poco usadas al área de **swap** en disco. Como el disco es miles de veces más lento que la RAM, el sistema se percibe lento. Si ni la swap alcanza, el *OOM killer* termina el proceso que más memoria ocupa.

3.3 Carga del sistema

El *load average* (`/proc/loadavg`) es el promedio de procesos en estado ejecutable o en espera no interrumpible (normalmente E/S de disco) durante 1, 5 y 15 minutos. Un valor mayor al número de núcleos indica que hay procesos esperando CPU o disco.

4. Desarrollo

4.1 Script de monitoreo

El script `Codigo/salud_sistema.sh` realiza los siguientes pasos:

1. Define un PATH completo, porque cron ejecuta las tareas con un entorno mínimo.
2. Crea la carpeta de registros con `mkdir -p`.
3. Obtiene RAM y swap con `free -m`, disco con `df -h "$HOME"`, carga con `/proc/loadavg` y los 5 procesos de mayor consumo con `ps --sort=-%mem`.
4. Escribe un bloque con fecha y hora al final del log (`>>`) y agrega una línea de **ALERTA** si la RAM supera el 80 % o el disco el 85 %.

4.2 Programación de la tarea

```
$ cd Crontab/Codigo
$ chmod +x *.sh
$ ./instalar_cron.sh
$ crontab -l
*/2 * * * * /home/hector/.../Crontab/Codigo/salud_sistema.sh
$ systemctl status cron
$ tail -f ~/salud_sistema/salud_sistema.log
```

4.3 Ejemplo de una entrada del registro

```
===== 2026-09-29 10:14:00 =====
Encendido      : up 1 hour, 3 minutes
Carga (1/5/15): 1.12 0.87 0.64
RAM            : 5210 MiB usados / 7822 MiB (71%) - disponible 2268 MiB
Swap          : 312 MiB usados / 2047 MiB
Disco ($HOME) : 61G usados / 98G (66%) - libre 32G
Procesos que mas memoria usan: ...
```

Nota: los valores son ilustrativos. Las capturas en Terminal/capturas/ muestran los datos reales del equipo.

5. Análisis reflexivo del comportamiento del sistema operativo

5.1 Escenario de reposo contra escenario de carga

Para interpretar el registro se comparan dos momentos: el sistema recién iniciado (solo escritorio y una terminal) y el sistema de uso cotidiano (navegador con muchas pestañas, reproductor de música, editor de código y varias terminales).

Indicador	Comportamiento esperado al aumentar la carga	Justificación teórica
RAM disponible	Disminuye notablemente	Cada pestaña del navegador es un proceso aislado (<i>sandbox</i>) con su propio motor de JavaScript y renderizado. El aislamiento es

		más seguro, pero duplica estructuras en memoria.
Swap	Empieza a usarse cuando la RAM disponible es baja	El kernel expulsa páginas inactivas (por ejemplo, pestañas en segundo plano) al disco para dar espacio a los procesos activos.
Carga promedio	Sube en picos al abrir programas	Más procesos compiten por la CPU, y la lectura de binarios y bibliotecas desde el disco produce esperas de E/S.
Top de procesos	Dominan firefox/chrome, code, gnome-shell	Son aplicaciones con motores gráficos y de scripts pesados. gnome-shell compone todas las ventanas en pantalla.
Disco	Crece lentamente	Cachés del navegador, paquetes descargados, <i>snaps</i> y el propio log. El cambio se ve en horas o días, no en minutos.

5.2 Reflexión

El ejercicio muestra que un porcentaje alto en la columna *used* no significa necesariamente un problema, porque Linux aprovecha la RAM libre como caché. El indicador que realmente predice lentitud es la caída de la memoria disponible acompañada de uso creciente de swap. Entender esta diferencia evita malinterpretar las herramientas de monitoreo.

También se comprobó la ventaja de delegar la periodicidad al sistema operativo. Un bucle `while true; sleep` (como el demonio de la práctica anterior) mantiene un proceso vivo todo el tiempo. Con `cron` no existe ningún proceso del monitor entre mediciones: el demonio `cron`, que ya estaba en ejecución, crea el proceso solo cuando hace falta. Así la herramienta de observación casi no altera lo que está midiendo.

Por último, el entorno reducido de `cron` (sin el `PATH` ni las variables de la sesión gráfica) evidencia que los procesos heredan su entorno del proceso padre. El script funciona en la terminal porque hereda el entorno de `bash`, pero bajo `cron` hereda el del demonio. De ahí la importancia de las rutas absolutas y de declarar las variables necesarias.

6. Conclusiones

- `cron` permite automatizar tareas periódicas de forma eficiente y confiable, y queda registrado en `/var/log/syslog`.
- La memoria *available* y la swap son los indicadores más fieles de la presión de memoria en Linux.
- Las aplicaciones de escritorio modernas, en especial los navegadores con arquitectura multiproceso, son las principales consumidoras de RAM en un equipo de estudio o programación.
- Un historial periódico permite pasar de percibir la lentitud a explicarla con datos, y decidir qué cerrar o cuándo liberar espacio.

7. Referencias

- `man 5 crontab`, `man 8 cron`, `man 1 free`, `man 5 proc`.
- Silberschatz, A., Galvin, P. y Gagne, G. *Operating System Concepts*, 10.^a ed. Wiley.
- Documentación del kernel Linux: */proc/meminfo* y administración de memoria.